

AI AGENT FOUNDATIONS

Study Guide — How a Single Agent Plans, Remembers, and Invokes Tools

Week 5 · Day 1 · Prepared by Rajesh Pasham

Program Context

Item	Detail
Session Type	Concepts and architecture lecture — no live Palantir platform work today
Platform	Individual Palantir Developer Tier accounts, in continuous use since Week 1, Day 3
This Week's Lab (Day 3)	Multi-Agent Insurance Workflow — orchestrated agents that collaborate, invoke Actions and Functions, and write back to your Ontology
Skill Gained This Week	AI Agent Engineer — the fifth rung of the program's Executive Skills Matrix

Where Today Fits

Week 2 gave every batch a Claims Copilot that could retrieve knowledge and answer questions. Week 3 gave it hands: Actions and Functions that write back to the Ontology, with a human approving every step. Week 5 is where those two things merge — a system that plans its own sequence of tool calls, decides which Action or Function to invoke, and only pauses for a human where your governance model says it must.

This is not new theory in isolation — it is Weeks 2 through 4, recombined. The Claims Copilot's retrieval (Week 2), the Ontology Actions and Functions it can now call (Week 3), and the document-derived knowledge it can reason over (Week 4) are exactly the building blocks an agent orchestrates. Keep Weeks 2–4's exercises open as your reference for what each tool actually does.

Learning Objectives

By the end of today, you will be able to:

- Explain what makes something an “agent” rather than a copilot
- Describe the plan → act → observe loop, and why an agent re-plans after every tool call
- Distinguish short-term working memory from persisted long-term memory
- Explain how Week 3's Functions and Actions become an agent's tools
- Describe event-driven triggers and microservices-style agent isolation
- Name the seven insurance agents and your batch's two-agent focus for Day 3

Topic 1 — From Copilot to Agent

1. A Quick Recap: What Week 2's Copilot Actually Did

- **Retrieval, then generation** — Semantic search pulled relevant Ontology objects and document chunks; the LLM used them as context to produce one grounded answer.
- **A single request → single response cycle** — an adjuster asked a question, the Copilot answered it; the interaction was complete the moment a response was returned.
- **No memory of its own decisions** — the Copilot didn't track what it had already told the adjuster, or whether that answer led to any downstream action.

2. What Made It Not Yet an “Agent”

A grounded answer is not the same as an executed plan.

- **It answered one question at a time** — the Claims Copilot retrieved relevant knowledge with Semantic search and produced a single grounded response; the workflow began and ended in one exchange.
- **It never decided what to do next on its own** — any writeback that followed (Week 3's Ontology Actions) was triggered by a human choosing to submit it; the Copilot proposed an answer, it did not choose or sequence actions.
- **Nothing tracked whether the goal was actually achieved** — there was no loop checking the outcome of a step and adjusting course; each interaction was self-contained, with no notion of an ongoing objective.

3. Defining an Agent

Autonomy over a sequence of steps toward a goal, not a single response.

- **Goal-directed, not turn-directed** — an agent is given an objective (“validate this claim's coverage and flag anomalies”), not a single prompt, and determines how many steps that takes.
- **It chooses its own tools** — given a goal and a set of available Functions and Actions, the agent decides which to call, in what order, based on what it learns along the way.
- **It re-plans as it goes** — after each tool call, the agent observes the result and decides the next step, closer to how a human adjuster works a claim than a fixed checklist.

4. Copilot vs. Workflow vs. Agent

Three patterns you've now built — knowing which one you're looking at matters.

Dimension	Copilot (Week 2)	Fixed Workflow (Week 3)	Agent (Week 5)
Trigger	A human asks a question	A defined event, every time	A goal, decomposed at runtime
Step sequence	One retrieval + one answer	Fixed, identical every run	Decided and revised by the agent
Tool choice	None — retrieval only	Predetermined by the workflow author	Chosen by the agent from what's available
Adapts mid-run	No	No	Yes — replans after each observation
Stops when	One answer is returned	The last defined step completes	The goal is met, or a human is needed

5. The Reasoning Loop: Plan → Act → Observe

This loop is what separates an agent from a workflow. A fixed AIP Logic workflow from Week 3 executes the same steps every run; an agent decides its steps at runtime, based on what each tool call returns — including deciding to stop and hand off to a human, per your batch's governance model.

- **Plan** — decide the next step toward the goal, given everything known so far
- **Act** — invoke a tool: a Function call, an Ontology Action, or a Semantic search
- **Observe** — read the result of that call, and update what the agent knows
- **Repeat** — until the goal is met, or the agent determines a human must decide

6. Walking the Loop With a Real Claim

A first notice of loss, worked step by step by a Claims Agent:

1. Goal received: “Assess FNOL-2201 and recommend next action”

The agent is not told which checks to run — only the outcome expected of it.

2. Plan: check policy status first

The agent reasons that coverage validity gates everything else, so it plans that step first.

3. Act: call the Coverage Validation Function

The same Function type built in Week 3 — invoked here by the agent's own choice, not a fixed sequence.

4. Observe: policy is active, but claim amount exceeds the policy's rating tier

This observation changes the plan — the next step is no longer a routine approval path.

5. Plan again: route to the Fraud Detection Agent before proceeding

The agent recognizes this pattern warrants collaboration — a preview of Day 2's multi-agent orchestration.

6. Stop: hand off to a human adjuster for the high-value referral

The agent recognizes the boundary set by your batch's governance plan and pauses rather than acting further.

7. When an Agent Should Stop and Ask a Human

Autonomy is not the same as unlimited authority.

- **A monetary threshold is crossed** — the same `isHighValueClaim` / `isHighRiskApplication` boolean pattern from Week 1 can gate not just a security policy, but an agent's authority to act without approval.
- **The agent's confidence in its own plan is low** — an agent that cannot resolve conflicting signals (a valid policy but a suspicious claim pattern) should escalate rather than guess.
- **The action is irreversible or hard to reverse** — settlement authorization and policy cancellation are exactly the kind of writebacks Week 3's rollback-strategy discussion was preparing you for.

Topic 2 — Agent Architecture

1. The Three Parts of Every Agent

Whatever it automates, an agent needs all three — none of them optional.

- **Planning** — how the agent turns a goal into an ordered (and revisable) sequence of steps.
- **Memory** — what the agent carries forward, within a single run, and, deliberately, across runs.
- **Tool invocation** — how the agent actually does something, by calling a Function, an Action, or a search, and reading back the result.

2. Planning: Decomposing a Goal Into Steps

- **A goal is not a step-by-step recipe** — “determine whether this claim is covered” could mean checking the policy, checking exclusions, checking claim history, or all three; the agent decomposes the goal into a candidate sequence.
- **Planning is revised after every observation** — if checking the policy reveals a lapsed payment, the plan changes; the next step is no longer coverage validation, it's a referral, exactly as Week 1's Referral object type anticipated.
- **Planning is bounded by what tools exist** — an agent can only plan around the Functions and Actions actually exposed to it; the same access-control discipline from Week 1's RBAC/ABAC governs which tools an agent may even see.

3. Two Planning Strategies

Strategy	How It Decides	Best Suited To
Reactive (single-step) planning	Chooses the single next best action given the current state, without projecting further ahead	Well-understood, low-ambiguity tasks — routing a document by its classification
Deliberate (multi-step) planning	Projects a candidate sequence of several steps before acting, and revises the whole sequence as new information arrives	Ambiguous, multi-signal tasks — assessing whether a claim is fraudulent

4. Memory: What an Agent Carries Between Steps

Memory Type	What It Holds	Governance Implication
Short-term (working) memory	What the agent has learned so far in this run — the claim it's investigating, the coverage result it already retrieved	Scoped to a single run, discarded when the run ends — the default for this program
Long-term (persisted) memory	State an agent deliberately keeps across runs — for example, a pattern seen across multiple prior claims	A design decision with real governance cost: what's retained, where it's stored, who can read it, how long it lives

Default to short-term unless the use case demands otherwise. Persisted memory is a deliberate design choice, not a free upgrade — Week 6's LLMOps and governance topics return to these questions in depth.

5. Memory in Practice: A Fraud Agent Example

- **Without long-term memory** — each claim is assessed independently; a single suspicious claim from a policyholder is evaluated on its own facts alone, with no view of that policyholder's claim history.

- **With long-term memory, deliberately scoped** — the Fraud Detection Agent retains a policyholder-level pattern (three claims in six months from the same repair shop), but only that pattern, retained under an explicit retention policy, not an open-ended transcript of every prior interaction.
- **The governance question either way** — who approved this retention, how long is it kept, and can the policyholder's data be purged on request; the same questions Week 1's Markings and security policies were built to answer for structured data.

6. Tool Invocation: Functions and Actions as an Agent's Tools

- **A Function is a tool the agent can call** — the same AIP Logic Functions from Week 3; an agent invokes one the way a workflow step did, but the agent decides when and whether to call it.
- **An Action is how an agent writes back** — Ontology Actions remain the only path to a committed change; an agent proposing an Action still goes through the same Object Explorer submission and, where your governance model requires it, the same human approval.
- **Tool invocation is where autonomy meets governance** — giving an agent access to a tool is a security-policy decision, using the same RBAC/ABAC and Marking patterns from Week 1; an agent should only ever see the tools its role actually needs.

7. Tool Selection: How an Agent Chooses the Right Tool

- **Every tool needs a clear description** — a Function or Action exposed to an agent carries a description of what it does and what inputs it needs; an agent reasons over these descriptions the same way it reasons over retrieved knowledge.
- **Ambiguous tool names cause wrong calls** — two Functions both named “UpdateStatus” force the agent to guess; the same naming discipline you used for Week 1's Object Types and properties applies directly to tools.
- **Fewer, well-scoped tools beat many overlapping ones** — an agent with ten narrow, clearly-described tools plans more reliably than one with three overloaded, multi-purpose ones.

8. Guardrails: Bounding What an Agent Can Do

- **Tool-level access control** — an agent only ever sees the Functions and Actions its role is granted; the same RBAC/ABAC model from Week 1, applied to tools instead of objects.
- **Approval gates on irreversible actions** — the human-in-the-loop approval pattern from Week 3 doesn't disappear; it becomes the condition that stops an agent's plan and waits, for exactly the actions your governance plan marks as sensitive.
- **Bounded run length and scope** — an agent given no limit on how many steps it can take, or how broad its goal is, is a runaway-cost and runaway-risk problem; the same discipline as bounding a pipeline's compute in Foundry.

9. Error Handling for Agents

- **Distinguish a retryable failure from a hard stop** — a transient search timeout can be retried; a Function returning “policy not found” is a signal to change the plan, not to retry the same call.
- **Compensating actions, not silent rollback** — the same compensating-transaction pattern from Week 3's writeback discussion applies here; an agent that partially completes a multi-step plan needs a defined way to undo or flag what it already did.
- **Every failure is logged, not swallowed** — an agent that fails quietly is worse than a workflow that fails loudly; Week 6's observability and audit-trail topics depend on every tool call, success or failure, being recorded.

Topic 3 — Enterprise Agent Design Patterns

1. Event-Driven Agents: What Triggers a Run

- **Object-change events** — a new Claim object created, or a Risk Assessment's status changing to "Referred"; the same Ontology events your Automate discussions in Week 3 were building toward.
- **Scheduled events** — a Policy Recommendation Agent that runs nightly against policies approaching renewal, rather than waiting for a person to ask.
- **Upstream agent events** — one agent's output becoming another agent's trigger; the Fraud Detection Agent's flag becoming the event that wakes the Compliance Agent, previewing Day 2's collaboration model.

2. Microservices-Style Agent Isolation

- **Each agent owns its scope, and only its scope** — a Claims Agent doesn't need underwriting data any more than a claims microservice would need direct access to an underwriting database.
- **One agent's failure doesn't take down another** — if the Fraud Detection Agent's model needs retraining, the Underwriting Agent keeps running; the same isolation benefit a microservices architecture gives application services.
- **Agents version and deploy independently** — updating the Compliance Agent's rules shouldn't require redeploying the Claims Agent; independent release cycles are a design goal, not an accident.

3. Enterprise Design Patterns Applied to Agents

Pattern	Where You've Seen It Before	How It Applies to Agents
Event-driven architecture	Week 3 — Automate triggers on Ontology events	An agent run is triggered by an event, not polled for on a schedule by default
Microservices-style isolation	General enterprise architecture practice	Each agent is scoped, deployed, and secured independently
Least-privilege access	Week 1 — RBAC / ABAC on objects and properties	An agent sees only the tools and data its role requires
CQRS (read vs. write separation)	Week 3 — Ontology reads vs. Action writebacks	An agent's reasoning (read) is separated from its committed writeback (Action submission)

4. Agent Studio: Where These Concepts Live in AIP

- **Agent Studio configures the reasoning loop** — the goal, the available tools, and the model behind an agent are configured here; conceptually the same Use LLM configuration from Week 2's AIP Logic blocks, extended with tool access.
- **Tools are the same Functions and Actions you already built** — nothing new needs to be created for an agent to use a Week 3 Function; it is exposed to the agent as an available tool, with its existing description and permissions.
- **Wednesday's lab configures, it doesn't code from zero** — you will assign your batch's two agents in Agent Studio, wire them to existing Functions and Actions, and define their triggers; the mechanics, not the underlying platform, are new.

5. Orchestration Preview: The Pattern Wednesday Builds

One orchestrator, several specialized agents, one shared writeback path: an Orchestrator Agent sits above the Claims, Fraud, Underwriting, and Compliance Agents, each of which writes back through the same Ontology Actions path. Day 2 covers how the Orchestrator Agent decides which specialized agent to invoke and how results are combined — today's job is recognizing each box as something you already understand on its own.

Topic 4 — Insurance Agent Roles

1. Why Specialize Instead of Building One Large Agent

- **Easier to evaluate** — a Fraud Detection Agent's behavior can be scored on fraud-detection outcomes alone; Week 6's Evals become far harder to design against one agent handling five unrelated responsibilities.
- **Easier to secure** — a narrow agent needs a narrow set of tools and data access; the same least-privilege principle behind Week 1's object-level security policies, applied to what an agent is allowed to see and call.
- **Easier to isolate and deploy independently** — one agent's changes or failures don't require redeploying every other agent; the same reasoning behind a microservices architecture, applied to AI components instead of application services.

2. The Insurance Agents in This Program

Seven specialized roles. Wednesday's lab assigns two per batch, based on your use case.

Agent	Responsibility
Claims Agent	Coordinates claim intake through settlement recommendation
Underwriting Agent	Evaluates new-business applications against rating factors
Fraud Detection Agent	Flags anomalous claims for SIU review
Risk Assessment Agent	Scores applicant and policy risk on an ongoing basis
Compliance Agent	Checks a proposed action against regulatory and policy constraints
Policy Recommendation Agent	Recommends coverage, renewal, or retention offers
Customer Support Agent	Handles policyholder-facing queries and routes escalations

3. Agent Deep Dive: Claims Agent & Fraud Detection Agent (Batch 1)

- **Claims Agent — goal and tools** — move a claim from intake to a settlement recommendation, using the Coverage Validation Function, Claim History search, and Settlement Function, all built in earlier weeks.
- **Fraud Detection Agent — goal and tools** — flag anomalous claims before settlement, using a pattern-matching Function over claim history, Semantic search across prior SIU findings, and a Referral Action.
- **Where they collaborate** — the Claims Agent's plan includes a step that hands off to the Fraud Detection Agent whenever claim amount or pattern crosses a threshold, previewed in the walked-through scenario above.

4. Agent Deep Dive: Underwriting Agent & Risk Assessment Agent (Batch 2)

- **Underwriting Agent — goal and tools** — recommend an underwriting decision for a new application, using Rating Factor lookup, Risk Assessment retrieval, and an Underwriting Decision Action.
- **Risk Assessment Agent — goal and tools** — keep an applicant's risk score current as new information arrives, using the same `isHighRiskApplication` derivation pattern from Week 1, Day 4, now invoked by the agent instead of a fixed pipeline.

- **Where they collaborate** — the Underwriting Agent requests a fresh score from the Risk Assessment Agent before finalizing a decision, rather than trusting a stale, previously-computed value.

5. Agent Deep Dive: Compliance, Policy Recommendation & Customer Support Agents (Batches 3 & 4)

- **Compliance Agent (Batch 3)** — checks a proposed Fraud Detection Agent action against regulatory and internal policy constraints before it reaches a human, using policy-rule lookup and an approval-gate Action.
- **Policy Recommendation Agent (Batch 4)** — recommends coverage, renewal terms, or retention offers ahead of a policy's expiry, using renewal-offer lookup (the same retention_offers dataset from your program materials) and a Recommendation Action.
- **Customer Support Agent (Batch 4)** — resolves policyholder-facing queries directly, or routes to a human when the query exceeds its authority, using knowledge retrieval from Week 2's Copilot pattern and an escalation Action.

6. Your Batch's Agent Focus on Day 3

Batch	Use Case	Primary Agents
Batch 1 — India	Claims Operations	Claims Agent + Fraud Detection Agent
Batch 2 — India	Underwriting & Risk Assessment	Underwriting Agent + Risk Assessment Agent
Batch 3 — Europe	Fraud & SIU	Fraud Detection Agent + Compliance Agent
Batch 4 — Europe	Policy Servicing	Policy Recommendation Agent + Customer Support Agent

7. Common Pitfalls

- **Calling any Copilot an “agent”** — a retrieval-and-answer loop with no planning or tool invocation is still a copilot; reserve “agent” for a system that decides its own next step.
- **Giving an agent long-term memory by default** — persisted memory across runs is a deliberate design choice with its own governance cost, not a free upgrade; default to short-term, per-run memory unless a use case needs otherwise.
- **Skipping the plan → observe step after a tool call** — an agent that fires a Function and moves on without checking the result behaves like a fixed script with extra steps, not an agent.
- **Building one large agent instead of specialized ones** — a single agent trying to do claims, fraud, and underwriting together is harder to evaluate, harder to secure, and harder to debug than four narrow agents that collaborate.
- **Treating every event as worth waking an agent for** — an event-driven agent that fires on every minor Ontology change wastes compute and floods logs; trigger conditions need the same scrutiny as a security policy condition.
- **Assuming an agent's plan is always correct because it ran without error** — no error does not mean the right decision; Week 6's Evals exist precisely because an agent can complete a plan cleanly and still reach the wrong outcome.

8. Glossary

Term	Meaning
Agent	A goal-directed system that plans its own sequence of tool calls and re-plans based on results, rather than following a fixed script
Plan → Act → Observe Loop	The reasoning cycle an agent repeats: decide a next step, invoke a tool, read the result, decide again
Reactive Planning	Choosing the single next best action given current state, without projecting multiple steps ahead
Deliberate Planning	Projecting and revising a multi-step sequence before and during execution
Short-Term (Working) Memory	What an agent knows within a single run; discarded once the run ends
Long-Term (Persisted) Memory	State an agent deliberately retains across runs, with its own governance and access-control requirements
Tool Invocation	An agent calling a Function or an Action to gather information or make a change
Guardrail	A bound on what an agent may do — a tool it cannot see, an approval gate, or a run-length limit
Event-Driven Trigger	An object change, schedule, or upstream agent output that starts an agent run
Orchestrator Agent	An agent whose role is deciding which specialized agent to invoke and combining their results
Multi-Agent System	Several specialized agents collaborating on parts of a larger goal (previewed today, built Day 3)

9. Further Practice

Continue into Day 2, where these foundations extend into agent collaboration, event-driven orchestration in practice, and how the Orchestrator Agent combines specialized agents' results — the bridge into Day 3's hands-on Multi-Agent Insurance Workflow lab.